

Aplicativo Android que armazena e sincroniza perfis de equalização de som

Relatório de implementação de aplicativo Android para gerenciamento de perfis de áudio.

Links

Link do Repositório: [gitHub](#)

Link da Aplicação: [VehicleEqualizerApp](#)

Link do Vídeo: [Vídeo](#)

Objetivos De Aprendizagem

- Explicar o ciclo de vida das Activities;
- Configurar corretamente as permissões de segurança no arquivo Android Manifest;
- Utilizar princípios de Programação Orientada a Objetos no desenvolvimento Android;
- Integrar componentes de interface gráfica com a classe Java ou Kotlin;
- Configurar Intents para navegação entre Activities.

Introdução

Em um contexto geral, o sistema de áudio de um veículo se torna o principal meio de entretenimento para o condutor durante o seu período de permanência no veículo, seja ouvindo músicas, notícias ou até mesmo realizando chamadas via Bluetooth. Assim como existe gosto para comida, também existe gosto para músicas e dentro do gosto para músicas, existem aqueles mais exigentes que gostam de manipular o áudio. A principal ferramenta para isso em um infotainment de um veículo é o equalizador, que, por ajustes em determinadas frequências, permite ajustar a intensidade de graves, médios e agudos, por exemplo.

O problema que isso gera é que nem sempre aquela equalização agrada a todos. Se pensamos em um veículo compartilhado por mais de um condutor e cada um tendo um gosto musical diferente, toda vez que um deles entrasse no carro para ouvir música, seria necessário refazer a equalização para o seu perfil pessoal. Além disso, existem aqueles que, para cada estilo musical, mudam a equalização para melhor agradar seus ouvidos.

Objetivo

A principal função deste aplicativo é permitir a customização das configurações de equalização por diversos usuários ou até mesmo tipos diferentes de equalização do mesmo usuário conforme o gênero de música que esteja ouvindo. Ou seja, a ideia é que o aplicativo permita salvar diferentes perfis de equalização de acordo com o gosto do condutor do veículo.

Sumário

- [1. Ciclo De Vida das Activities](#)
- [2. Descrição Da Solução](#)

- 2.1. Uso Do Aplicativo
 - 2.1.1 Tela Inicial
 - 2.1.2 Tela de Edição
 - 2.1.3. Excluindo Um Perfil
- 3. Tecnologias Abordadas
- 4. Conclusão
- 5. Referências

1. Ciclo De Vida das Activities

No desenvolvimento de aplicativos Android, a **Activity** é um componente fundamental que representa uma única tela de interface do usuário. Seu comportamento e estado são controlados pelo **ciclo de vida da Activity**, que é gerenciado por um conjunto de callbacks que o sistema executa em momentos específicos (como quando a tela é criada, iniciada ou destruída)[1][2].

A figura 1 abaixo apresenta o fluxo do ciclo de vida de uma activity.

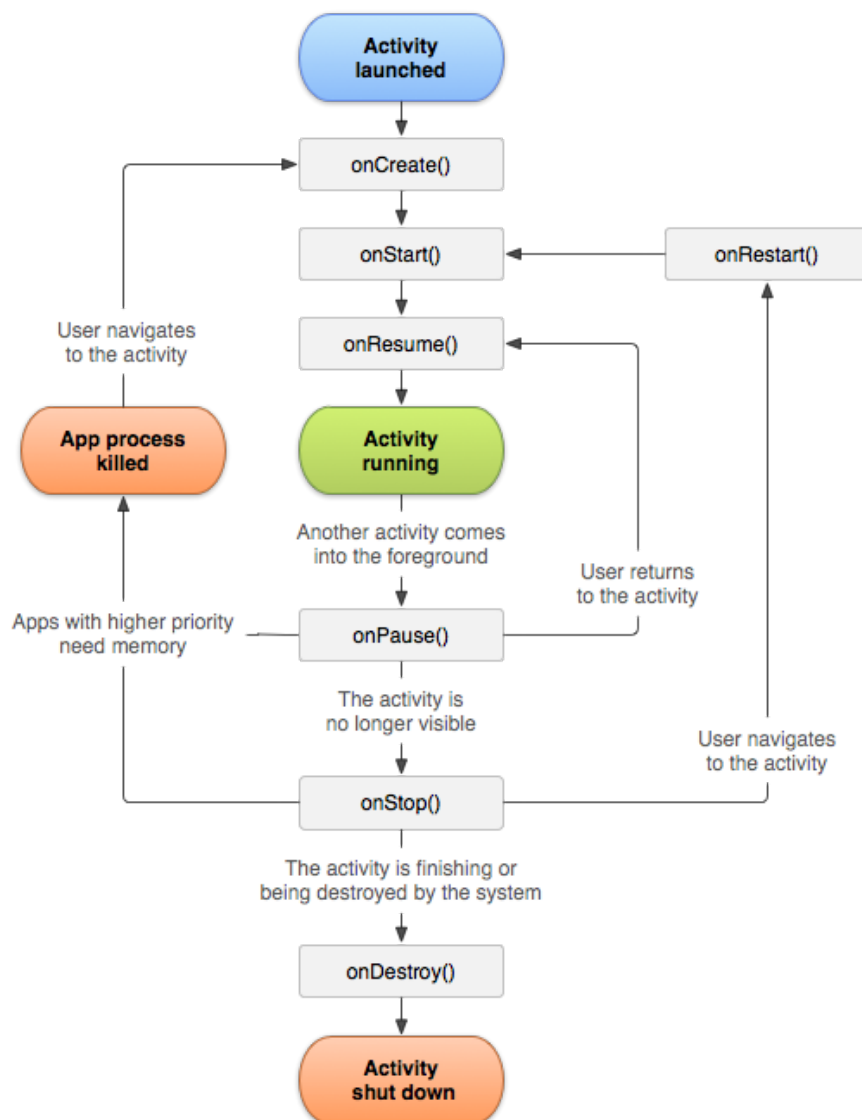


Figura 1: Ciclo de vida de uma Activity

Analisando a imagem anterior, temos os seguintes estados:

- **onCreate()**
 - A Activity é criada. É o ponto de entrada único para o ciclo de vida. Aqui, ocorre a inicialização da interface do usuário (UI) e os componentes essenciais.
- **onStart()**
 - A Activity se torna visível para o usuário.
- **onResume()**
 - A Activity está em primeiro plano e pronta para a interação do usuário. Este é o estado de "execução" da Activity.
- **onPause()**
 - A Activity está parcialmente visível, mas não está mais em primeiro plano. Este estado é chamado quando outra Activity aparece por cima (como um diálogo) ou quando o usuário está prestes a sair dela.
- **onStop()**
 - A Activity não está mais visível para o usuário.
- **onDestroy()**
 - A Activity está sendo destruída e finalizada. É o último estado do ciclo de vida, onde todos os recursos devem ser liberados.
- **onRestart()**
 - A Activity que estava em estado de **onStop()** é chamada para ser reiniciada, passando para o estado de **onStart()** em seguida.

2. Descrição Da Solução

O aplicativo desenvolvido é capaz de armazenar perfis de equalização contendo configurações customizadas baseadas nos ajustes do usuário. A figura abaixo descreve o fluxo de funcionamento do aplicativo.

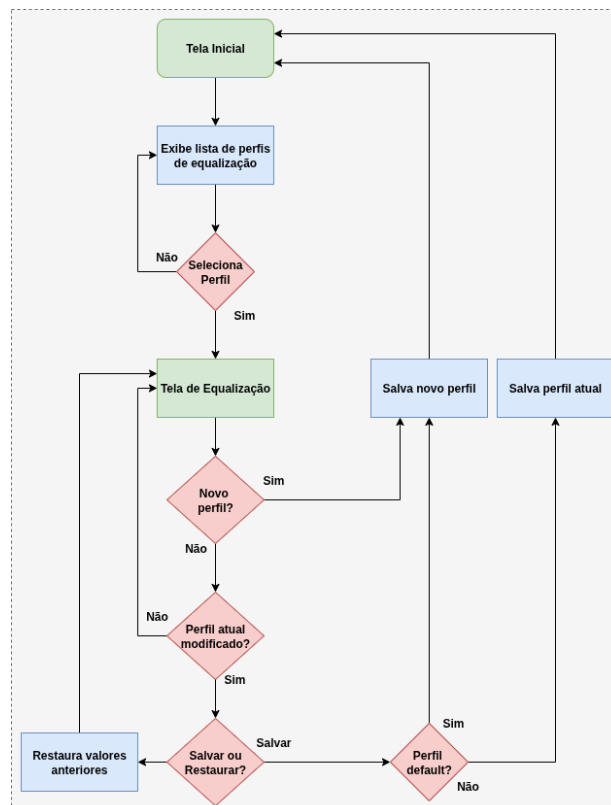


Figura 2: Fluxograma da solução

2.1. Uso Do Aplicativo

2.1.1. Tela Inicial

A tela abaixo é a tela inicial do aplicativo. Ao abrir o aplicativo, é exibida a tela de perfis. Caso não tenha nenhum perfil cadastrado, o único perfil que será exibido é o perfil **Default**.

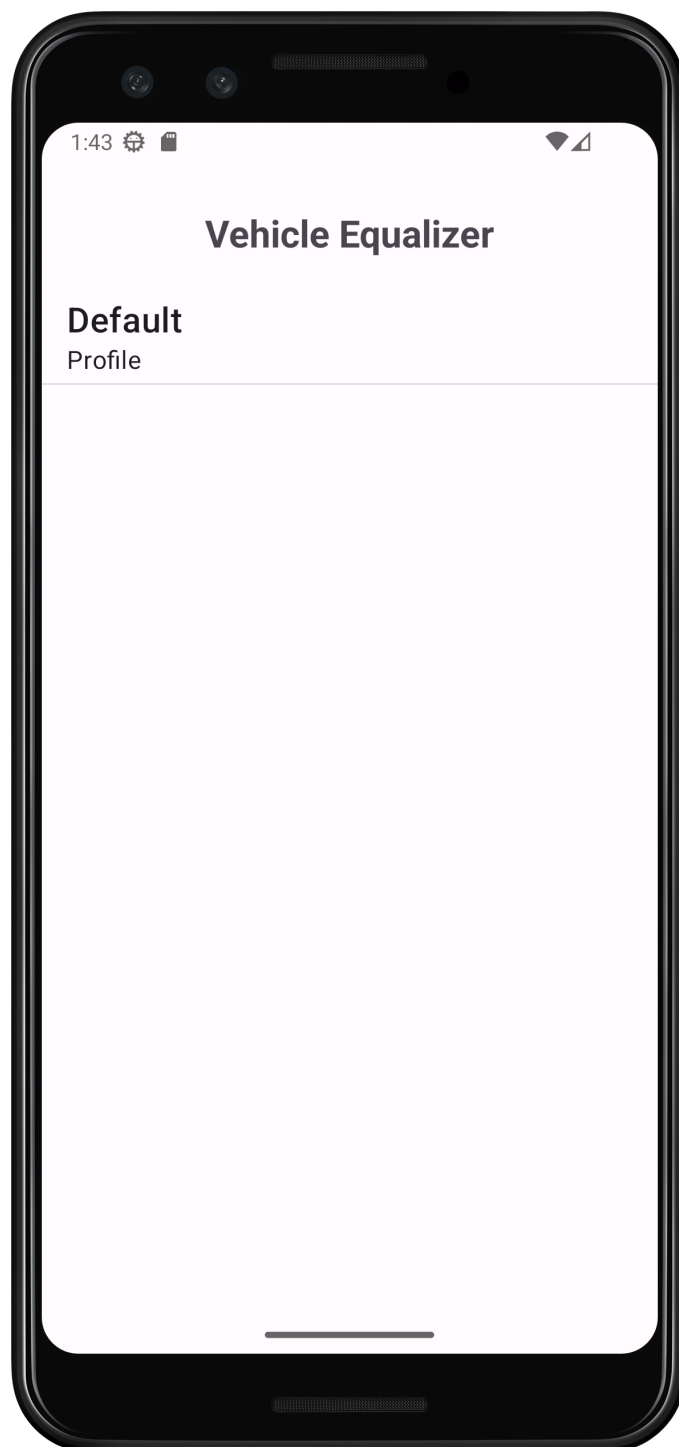


Figura 3: Tela inicial com o perfil Default

A tela seguinte exibe a mesma tela com vários perfis cadastrados.

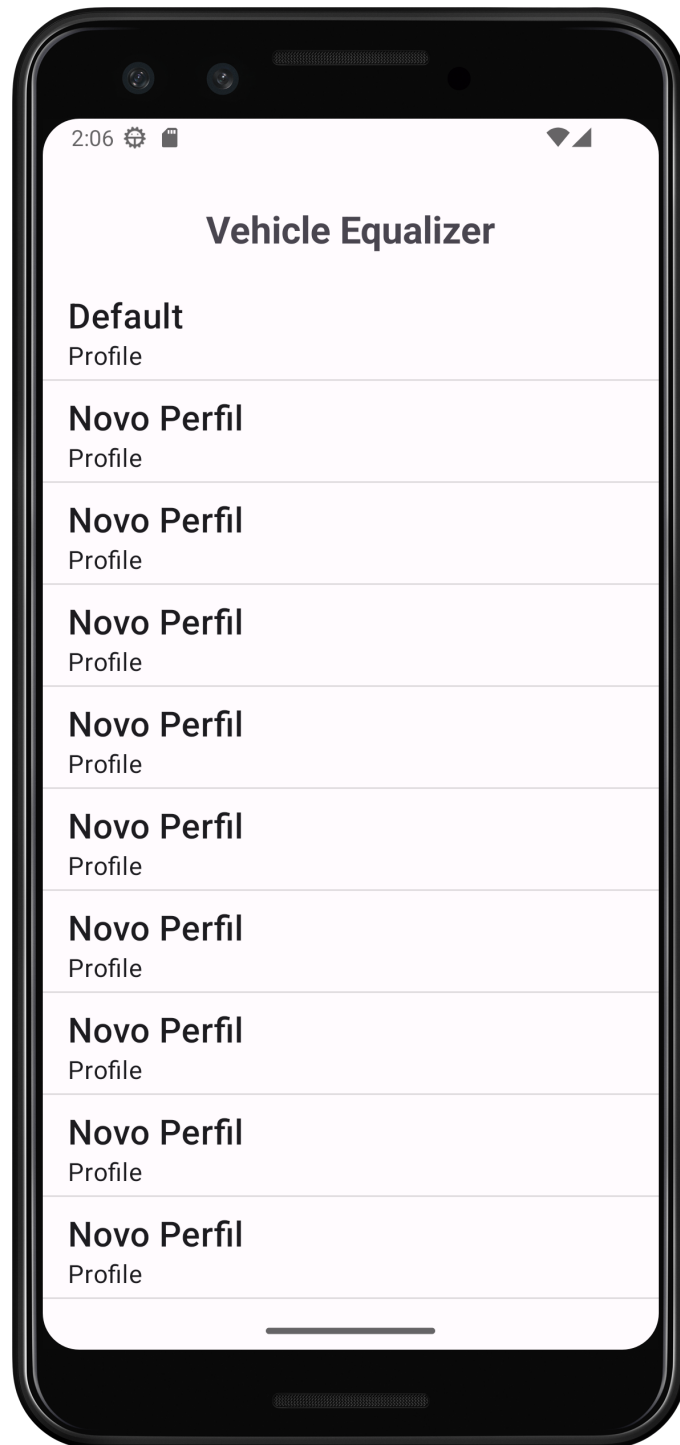


Figura 4: Tela inicial com vários perfis

Na próxima seção, veremos como editar e criar um novo perfil.

2.1.2. Tela de Edição

Para entrar na tela de edição, basta tocar em qualquer perfil da tela principal e a tela da figura abaixo será exibida. Um novo perfil sempre é criado a partir de um perfil existente, seja o perfil Default ou outro qualquer.

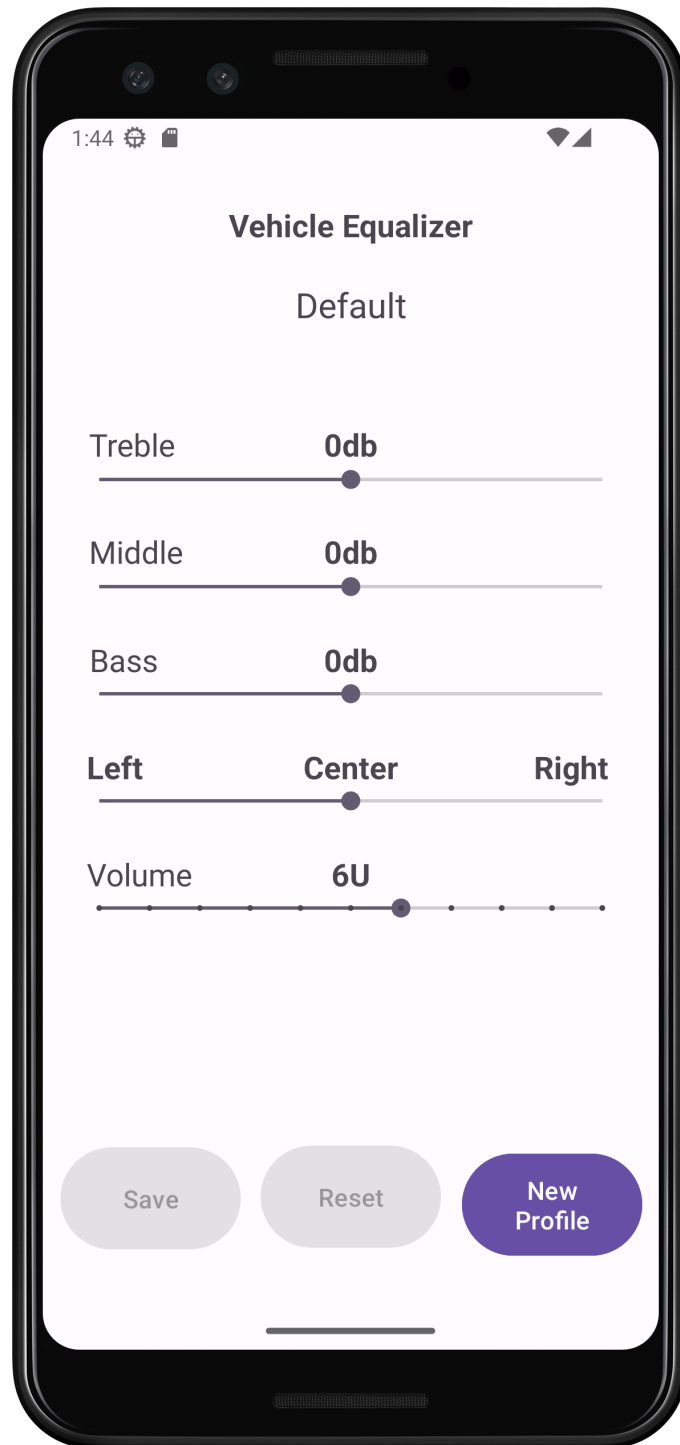


Figura 5: Tela de configuração

A tela de configuração possui os ajustes de áudio principal de um sistema de som veicular, cortes de frequências, graves, médios e agudos, além do ajuste de panorama e volume.

Na parte inferior da tela, é possível notar que os botões de **Salvar** e **Reset** não estão disponíveis. Estes botoes só se tornam utilizáveis quando alguma configuração é alterada.

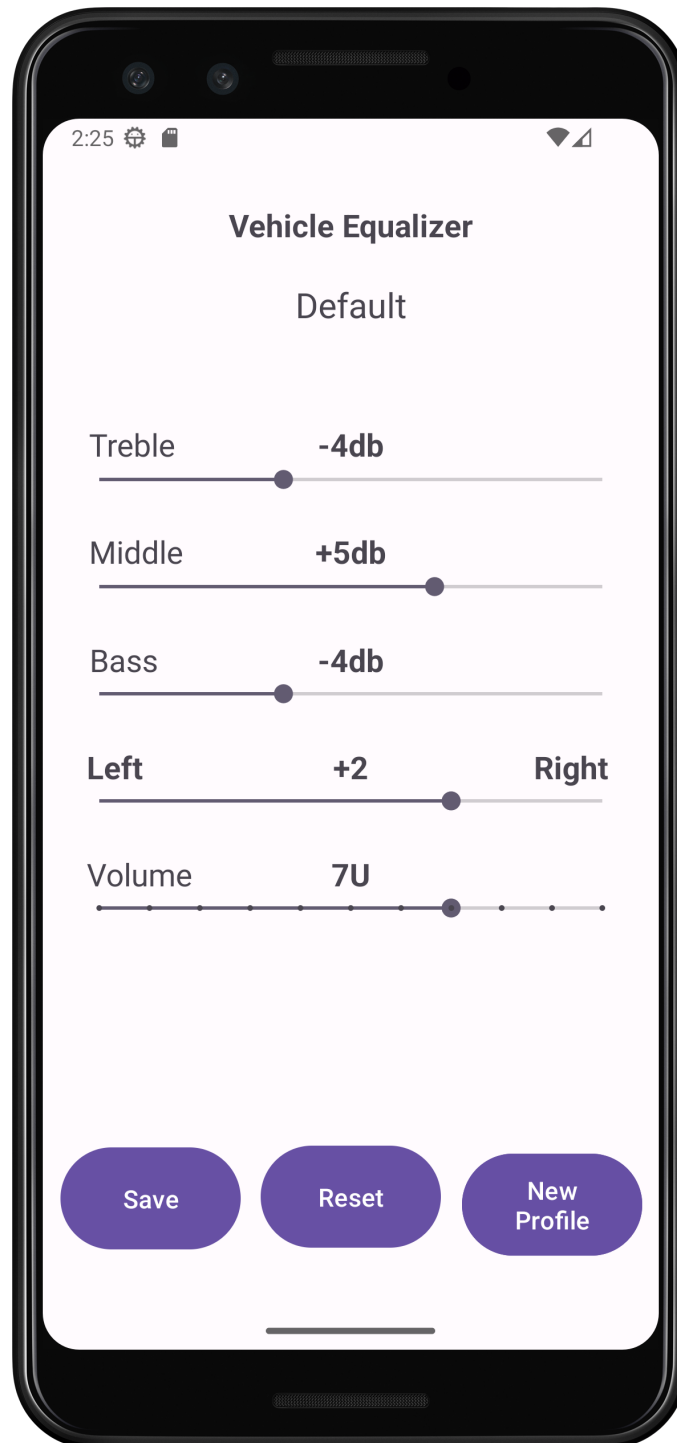


Figura 6: Alteração de configurações

A figura acima exibe os botões ativos após a alteração das configurações. a regra de negócio aplicada para esses botões é a seguinte:

- **Save:** Salva as configurações no perfil atual, exceto se o perfil atual for o perfil Default, neste caso, cria um novo perfil.
- **Reset:** Restaura para as configurações iniciais do perfil, caso ainda não tenham sido salvas.
- **New Profile:** Cria novo perfil.

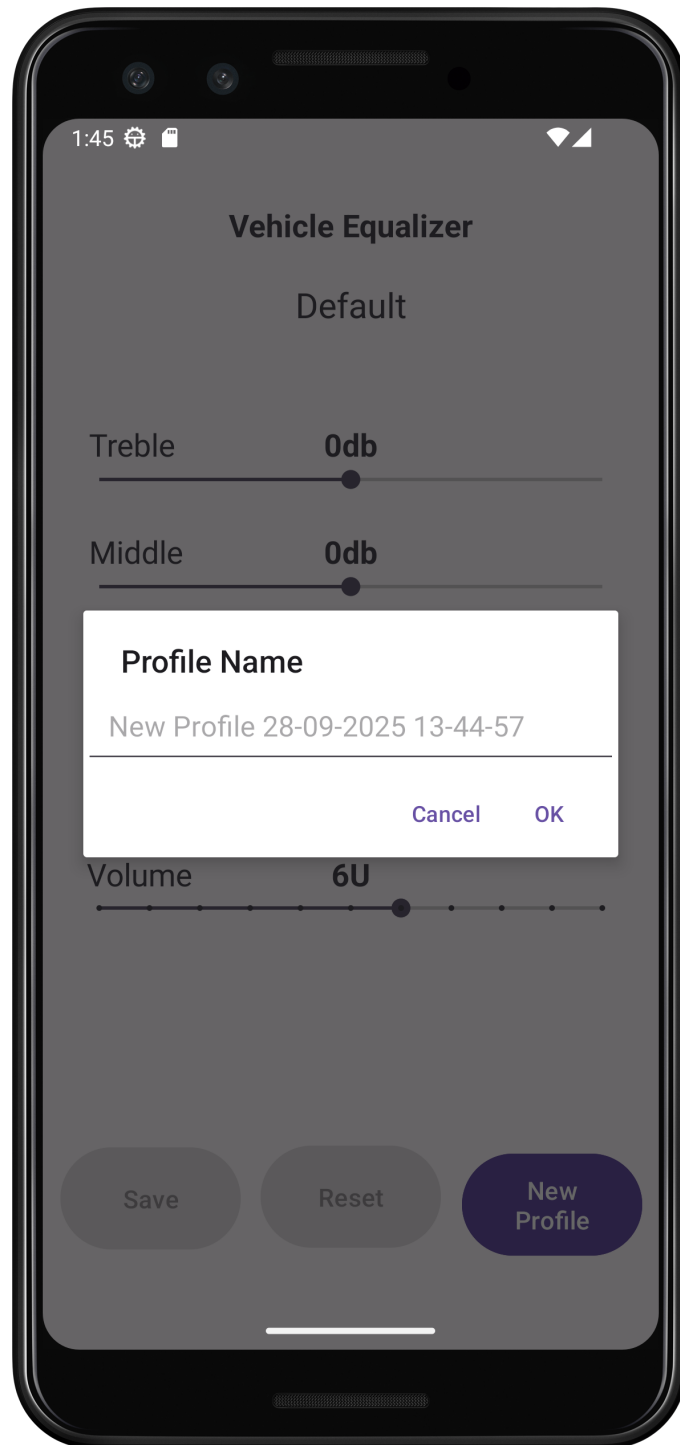


Figura 7: Nome do novo perfil

Ao apertar para criar um novo perfil, é exibido um **AlertDialog** com o nome padrão, caso nenhum nome seja digitado, ao apertar em Ok, o perfil é salvo e o aplicativo volta à tela inicial.

Além de voltar para a tela inicial, a mensagem de aviso é exibida na tela conforme a figura abaixo. A mesma também é exibida ao selecionar um perfil.

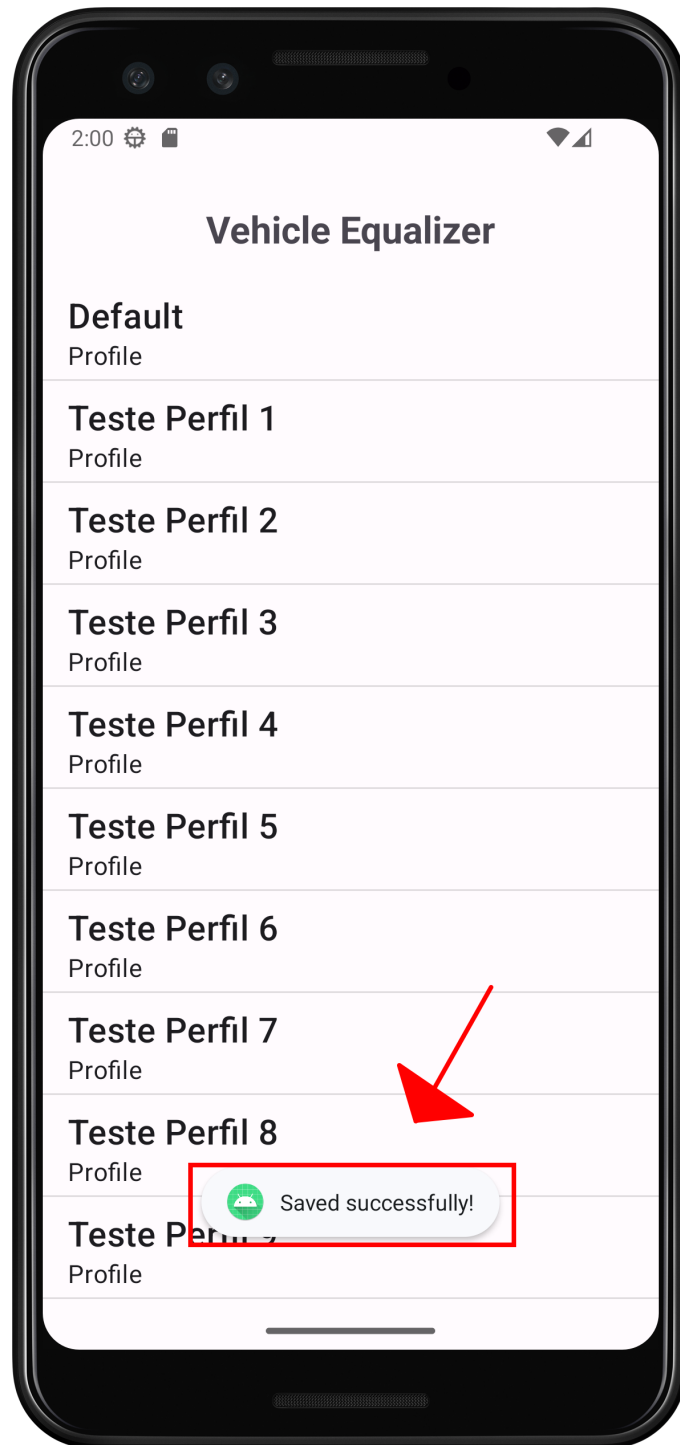


Figura 8: Perfil salvo

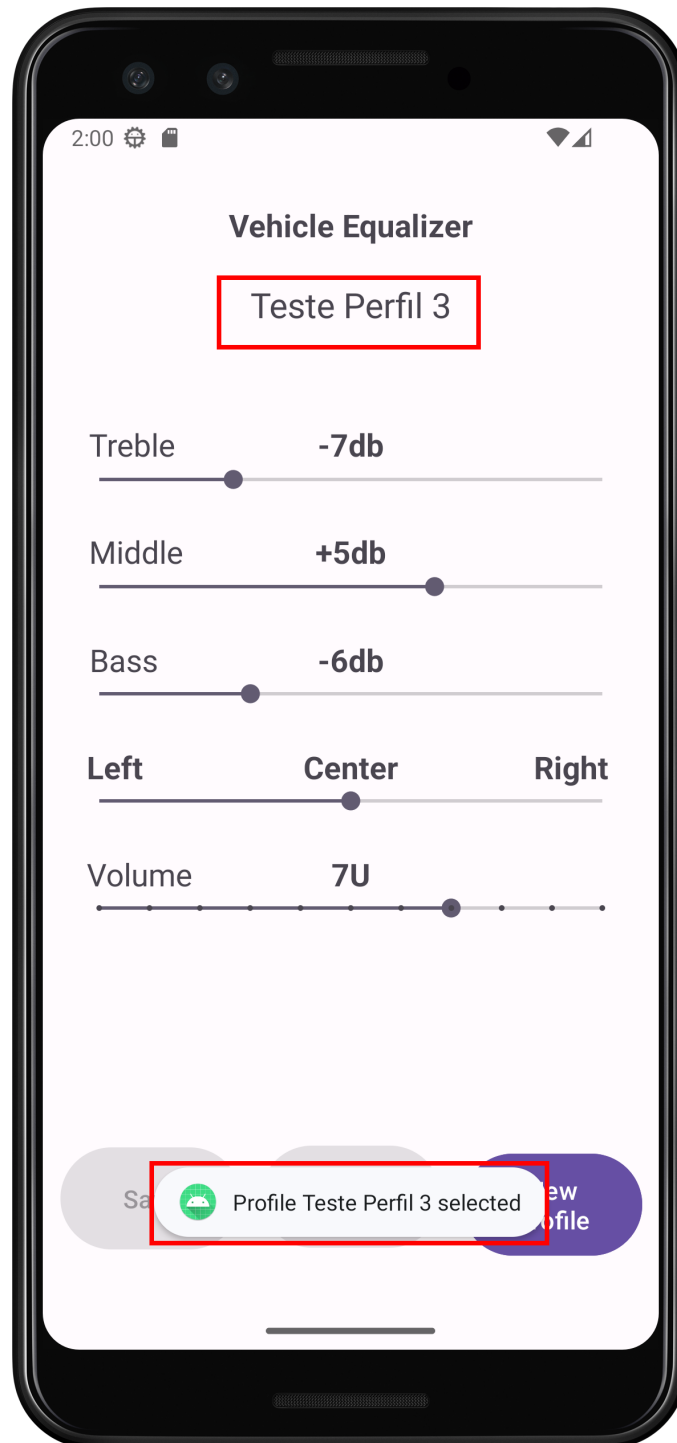


Figura 9: Perfil selecionado

2.1.3. Excluindo Um Perfil

Para excluir um perfil, é relativamente simples, basta ir para a tela inicial e segurar apertando sobre o perfil que deseja apagar e uma mensagem de alerta será exibida pedindo para confirmar.

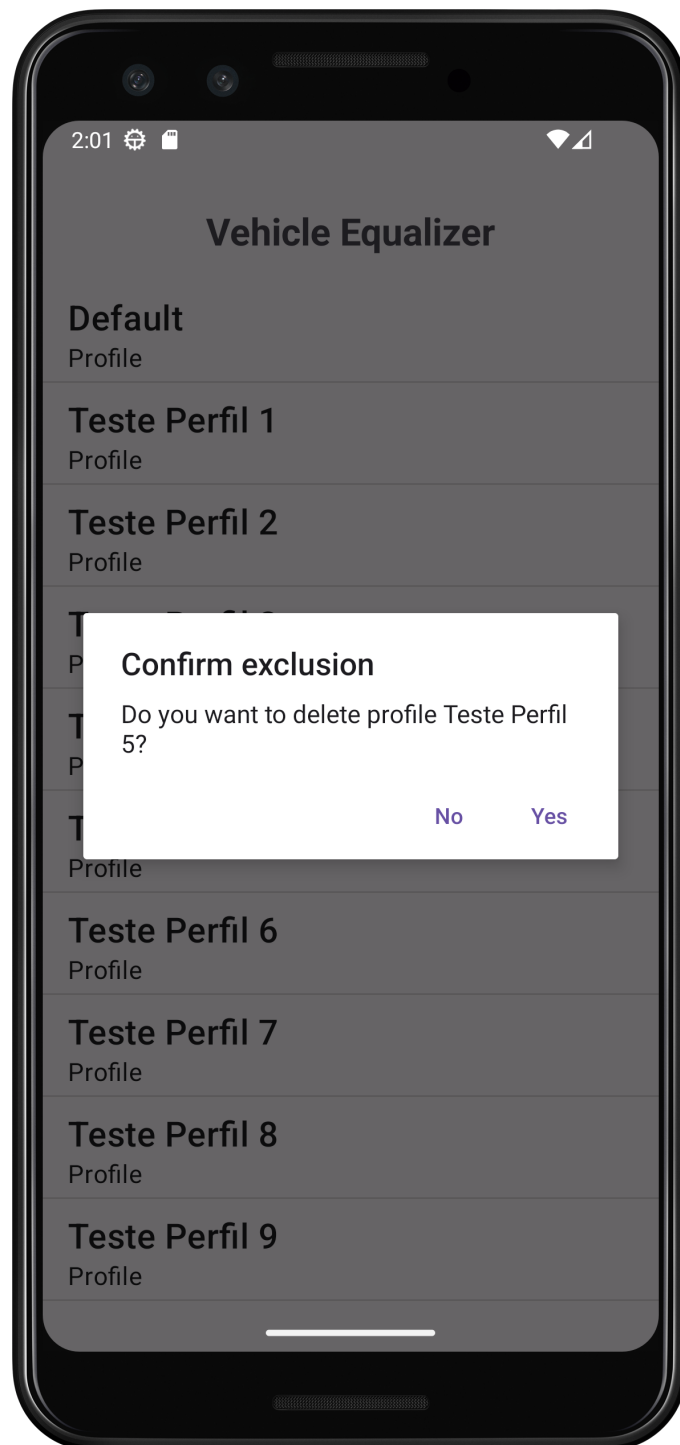


Figura 10: Apagando perfil

O perfil default não pode ser apagado, caso tente apagar, a seguinte mensagem aparecerá conforme figura abaixo.



Figura 11: Apagando perfil Default

3. Tecnologias Abordadas

O código do aplicativo foi implementado para seguir a arquitetura MVVM[3]. A principal característica da arquitetura MVVM é a divisão de responsabilidades em três principais componentes:

- **Modelo:** Camada que representa os dados do aplicativo. Ela interage com fontes de dados, como bancos de dados ou APIs de rede.
- **Visualização(View):** Camada que gerencia os elementos da interface do usuário (IU) e seu layout. Ela exibe dados ao usuário e captura suas interações.

- **ViewModel:** Esta camada atua como intermediária entre o Modelo e a Visualização. Ela prepara os dados para a Visualização de forma consumível, manipula a lógica de negócios e expõe fluxos de dados observáveis à Visualização.

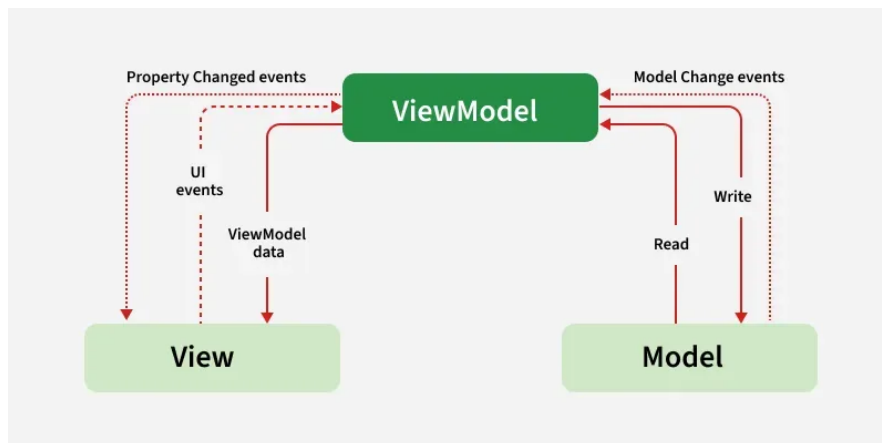


Figura 12: MVVM Architecture

A imagem abaixo apresenta a estrutura de pastas do projeto e, para cada arquivo, vou caracterizá-lo dentro das camadas da arquitetura MVVM.

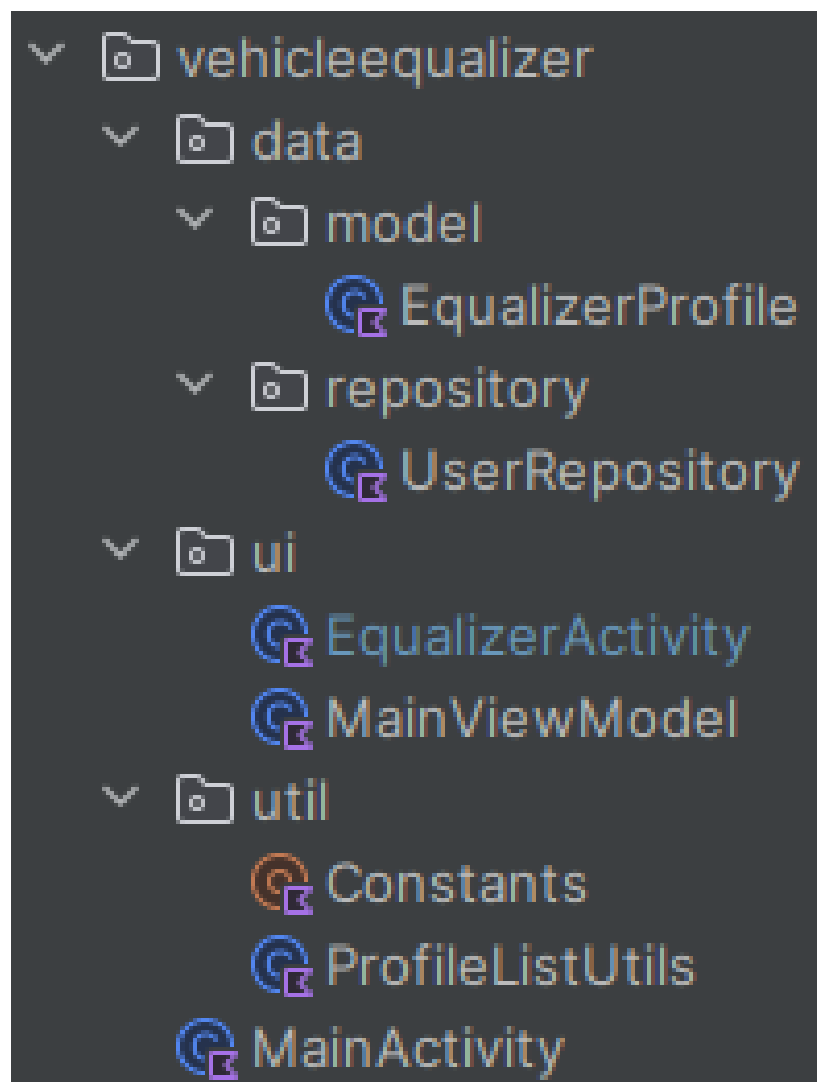


Figura 13: Estrutura do código fonte do aplicativo `VehicleEqualizer`

Esta é a classe principal do modelo de dados que representa o objeto do perfil de equalização. A anotação `@Parcelize` implementa de maneira implícita todos os métodos `Parcelable`. Na figura 13 também temos a classe `UserRepository` que está também dentro do pacote de dados, responsável pelo gerenciamento dos dados.

```
package com.leandromendes.vehicleequalizer.data.model

import android.os.Parcelable
import com.leandromendes.vehicleequalizer.util.Constants
import kotlinx.parcelize.Parcelize

@Parcelize
data class EqualizerProfile(
    var name: String = Constants.define.PROFILE_DEFAULT_NAME,
    var bassEqValue: Int = Constants.define.BASS_VALUE_DEFAULT,
    var midEqValue: Int = Constants.define.MIDDLE_VALUE_DEFAULT,
    var hiEqValue: Int = Constants.define.TREBLE_VALUE_DEFAULT,
    var balanceEqValue: Int = Constants.define.PAN_VALUE_DEFAULT,
    var masterVolValue: Int = Constants.define.VOLUME_VALUE_DEFAULT,
    var isSelected: Boolean = true
) : Parcelable
```

De maneira geral tenho a seguinte estrutura no meu aplicativo:

```
com
├── leandromendes
│   └── vehicleequalizer
│       ├── data # Contém classes relacionadas ao gerenciamento de dados:
│       │   │   ├── model # Define os modelos de dados usados no aplicativo.
│       │   │   │   ├── EqualizerProfile.kt
│       │   │   └── repository # Implementa o padrão de repositório para
│       │   │       manipular operações de dados.
│       │   │       ├── UserRepository.kt
│       │   └── MainActivity.kt
│       └── ui # Contém componentes de interface do usuário, como fragments
│           │   ├── EqualizerActivity.kt
│           └── viewmodel # Contém classes ViewModel que gerenciam os dados
│               relacionados à interface do usuário.
│               ├── MainViewModel.kt
│       └── util # Métodos utilitários
│           ├── Constants.kt
│           └── ProfileListUtils.kt
```

3.1. Testes Unitários

A classe `UserRepositoryTest` foi implementada para testar todos os métodos da classe `UserRepository` com o intuito de validar os principais métodos de gerenciamento de dados.

```
class UserRepositoryTest {
    private lateinit var userRepository: UserRepository

    private val defaultProfile = EqualizerProfile()
    private val newProfile1 = EqualizerProfile(
        name = "Profile_1",
        bassEqValue = Constants.define.BASS_VALUE_DEFAULT,
        midEqValue = Constants.define.MIDDLE_VALUE_DEFAULT,
        hiEqValue = Constants.define.TREBLE_VALUE_DEFAULT,
        balanceEqValue = Constants.define.PAN_VALUE_DEFAULT,
        masterVolValue = Constants.define.VOLUME_VALUE_DEFAULT,
        isSelected = false
    )

    @Before
    fun setUp() {
        userRepository = UserRepository()
    }

    // --- Initialization Tests ---
    @Test
    fun init_repositoryStartsWithOneDefaultProfile() {
        // GIVEN: The repository was initialized in @Before.

        // WHEN & THEN: Check that the list is not empty and has the
        // expected size (1)
        assertEquals(1, userRepository.allEqualizerProfiles.size)

        // Check if the first profile is the default
        assertEquals(defaultProfile.name,
            userRepository.getEqualizerProfile(0).name)
    }

    // --- Tests for addProfile ---
    @Test
    fun addProfile_addsNewProfileToListCorrectly() {
        // GIVEN: Repository with 1 default profile (from init)

        // WHEN: Add a new profile
        userRepository.addProfile(newProfile1)

        // THEN: The size of the list should be 2 and the new profile
        // should be in the last position
        assertEquals(2, userRepository.allEqualizerProfiles.size)
        assertEquals(newProfile1.name,
            userRepository.getEqualizerProfile(1).name)
    }

    // --- Tests for removeProfile ---
}
```

```

@Test
fun removeProfile_removesProfileByValidIndex() {
    // GIVEN: Add a second profile
    userRepository.addProfile(newProfile1)
    assertEquals(2, userRepository.allEqualizerProfiles.size) //
Verifica o GIVEN

    // WHEN: Remove the added profile (index 1)
    userRepository.removeProfile(1)

    // THEN: The size should return to 1
    assertEquals(1, userRepository.allEqualizerProfiles.size)

    // Check if the remaining profile is the default one
    assertEquals(defaultProfile.name,
userRepository.getEqualizerProfile(0).name)
}

@Test
fun removeProfile_doesNothingIfIndexIsInvalid() {
    // GIVEN: Repository with 1 default profile
    val initialSize = userRepository.allEqualizerProfiles.size

    // WHEN: Attempts to remove an invalid index (outside the upper
limit)
userRepository.removeProfile(99)
    // E: Attempts to remove an invalid (negative) index
userRepository.removeProfile(-1)

    // THEN: The size of the list should not change
    assertEquals(initialSize, userRepository.allEqualizerProfiles.size)
}

// --- Testes para updateProfile ---
@Test
fun updateProfile_replacesProfileAtValidIndex() {
    // GIVEN: Repository with the default profile at index 0
    val newName = "Updated Profile"
    val updatedProfile = EqualizerProfile(name = newName)

    // WHEN: Update profile in index 0
    userRepository.updateProfile(0, updatedProfile)

    // THEN: The name of the profile in index 0 should be the new name
    assertEquals(newName, userRepository.getEqualizerProfile(0).name)
    // The size of the list should remain 1
    assertEquals(1, userRepository.allEqualizerProfiles.size)
}

@Test
fun updateProfile_doesNothingIfIndexIsInvalid() {
    // GIVEN: Repository with 1 default profile
    val originalProfile = userRepository.getEqualizerProfile(0)
    val newName = "Invalid Test"

```



```

        val updatedProfile = EqualizerProfile(name = newName)

        // WHEN: Attempts to update an invalid index
        userRepository.updateProfile(99, updatedProfile)

        // THEN: The profile in index 0 should remain the original
        assertEquals(originalProfile.name,
            userRepository.getEqualizerProfile(0).name)
    }

    // --- Testes para getEqualizerProfile ---
    @Test
    fun getEqualizerProfile_returnsCorrectProfile() {
        // GIVEN: Adds the profile "Profile_1" to index 1
        userRepository.addProfile(newProfile1)

        // WHEN: Get the profile in index 1
        val retrievedProfile = userRepository.getEqualizerProfile(1)

        // THEN: The returned profile must have the name "Profile."
        assertEquals(newProfile1.name, retrievedProfile.name)
    }
}

```

Resultado da execução:

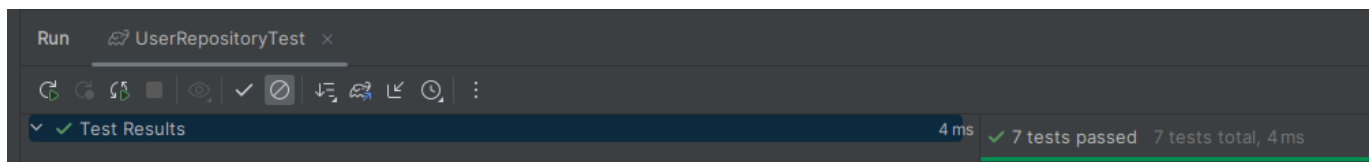


Figura 14: Testes Unitários

4. Conclusão

A aplicação ainda não possui conexão com o banco de dados, os dados são salvos em memória, mas acredito que o objetivo inicial foi atingido, que era a criação de perfis customizados de equalização.

Como próximos passos, botão para desativar e ativar o equalizador, além da implementação da conexão com o banco de dados.

Para o momento, não foi necessária a implementação de estrutura como.

- Processes and Threads
- Remote Procedure Calls

5. Referências

- 1 [Ciclo de vida da atividade](#)
- 2 [Ciclo de vida das Activities](#)
- 3 [MVVM \(Model View ViewModel\) Architecture Pattern in Android](#)